CS455

Deliverable 3

CodeClash

Ridham Divyakant Solanki (230853) Peeyush Agarwal (230749)
Srujan Sachin Bhirud (231039) Aritra Kolay (230192)
Om Chandrakant Chaudhari (230715)

`https://github.com/SrujanBhirud/CodeClash`

5 November 2026

Our repository is the one above. We merged the demo pack as pull request #48 on 5 November 2026. We did not rewrite earlier history. Pull requests #40–#46 remain as we merged them, with no review, and `check` stayed red on those merges until pull request #47 fixed the compile permission. We measured the tables below on 5 November 2026, on our compose stack `codeclash-srujan`, Docker Desktop 29.8.1, and Node 22.

The course list is the contents of this document. The decision log is the last section, in the same entry shape as the prompt log in our Deliverable 2.

Table 1: Where each required item is.

Required item	Section
Unit, integration, system, and adversarial tests	§1
Code coverage	§2
Concurrency	§3
Security and the threat model	§4
Performance, load, and the optimisation	§5
Failure and recovery	§6
Defect history, and five fixes	§7
Costing	§8
Timeline	§9
Deployment	§10
Requirements to Jira to GitHub to a test	§11
Jira dashboard and project report	§12
AI engineering log	§14; the bot is §13

1 Test reports

A test here is a check we can run again and get the same answer. We split them by what they are allowed to touch.

Table 2: What each required suite is for.

Suite	What it checks
Unit	One function, with no network. Scoring, a contest jumping states, quiz points, ratings, the agent tool gate, mutation edits, and the mail address parser.
Integration	The HTTP API against Mongo and Redis. Creating an account, confirming the email before a seat, registration and the waitlist, freeze, rejudge, the quiz, ratings, and starting a hardening run.
System	A browser. Sign in, open the arena, and check that the Paper background is `#f6f4ef`.
Adversarial	A hostile program in the sandbox, and a model reply that asks to publish. The program stays inside the container. The publish call is denied.

1.1 Unit

The shared package checks the rules a contest uses. A penalty counts only the rejections before the first accepted answer. A quiz answer scores full marks at the start and half marks at the deadline. A rating moves toward the contest result and never falls below 100. `publish`, and any tool name we do not recognise, is denied.

The agent checks that a mutant is a small edit of the reference, that a model reply which is not the allowed JSON is dropped, and that the pipeline still scores a suite when no model is configured.

Mail is five checks on the address, not a live send. Mailpit stays on port 1025 with no password. A relay on port 587 turns on STARTTLS and decodes a password that contains `@`. `smtps` is port 465. Port 25 is refused, because Oracle Cloud blocks it. The local-inbox link is returned only when the host is this PC. A real relay returns no local link, so the page tells the person to open their own email. On 6 November the public host used that relay: one register to our Gmail address returned `mailed: true`, with no code and no inbox link in the response.

1.2 Integration

`server/src/http/flows.test.ts` walks a problem from a draft, through the publish check, into the archive. It seats one user, waitlists the next, and promotes the first waiter when someone withdraws. It runs a contest, freezes the board, ends it, and publishes ratings. It also refuses a seat until the email is verified.

`auth.test.ts` checks the security routes. A participant receives 403 on an admin route, and the denial is written to the audit log. A used refresh token cannot mint a new session. An admin can change a role. A person can rename the account.

`domain.test.ts` is the concurrency file, described in §3. After the six-digit code, these three files were run again: 49 tests, pass. A signup against Mailpit returned `mailed: true`, and the code in that inbox verified with HTTP 200.

1.3 System

Playwright, `tests/e2e/core.spec.ts`, three tests, runs as the GitHub `e2e` job. It signs in, opens the arena, and reads the background colour. On this pack, `check` and `e2e` were green on run 42045118720.

1.4 Adversarial

`judge/src/isolation.test.ts` runs the escape programs. A fork bomb ends as RE or TLE. A read of a host file fails. A program that sleeps past the limit is TLE. A program that allocates past the limit is MLE. A program that prints without stopping is cut off. On the Hyderabad machine the fork bomb came back MLE: the 64 MB limit stopped it before the time limit, so that one assertion failed and the other escape programs matched.

`agent/src/engine/ai.test.ts` feeds the planner a reply that contains `publish`. The tool is denied and the denial is written to the audit log. A run that has spent its token cap returns `BUDGET_EXCEEDED` and does not call the model again.

`npm run lint`, `npm run typecheck`, `npm test`, and `npm run test:coverage` passed on the tree that includes the sandbox fix. The mail checks were added after that full run. They pass on their own, and the server row below includes them after we ran the server suite again.

The judge count includes the language suite and the batch suite. The language suite submits one correct program in Python, JavaScript, C, C++17, Java 21, and Go, and each one is accepted. The batch suite sends three programs through one container and expects AC, AC, WA. Before the compile-permission fix, those compiled languages came back as compile errors.

Table 3: Tests we ran, by workspace.

Workspace	Tests	Result
packages/shared	33	pass
agent	37	pass
server, including mail	86	pass
judge	22	pass

On 6 November we ran the same four workspaces on the Hyderabad machine in §10. Shared 33, agent 37, and server 86 passed, including the test that gives 200 seats to 400 users. Judge passed 21 and failed the fork bomb, as above. We did not re-run coverage, the k6 scripts, or the open burst of 400 registrations and 600 submissions. In the browser we signed in, opened "One number of FizzBuzz", submitted a Python solution, and the page said Accepted.

2 Code coverage

Our gate is 80% lines, functions, and statements in each workspace. We report branches. We do not fail the build on branches.

Table 4: Coverage on 5 November 2026, after CC-53.

Workspace	Lines	Functions	Statements	Branches
shared	94.99	95.65	94.99	85.58
agent	96.24	98.50	96.24	85.42
server	92.24	95.60	92.24	73.53
judge	94.31	87.50	94.31	83.72

Every workspace is above the gate. The lowest branch figure is the server, 73.53%. We left that as a report, not as a second gate.

3 Concurrency

`server/src/domain/domain.test.ts` registers 400 distinct users into a contest of 200 seats. The coverage run recorded that case at about 87 seconds. The result is exactly 200 seats and no duplicate seat.

The write is one filtered update. A seat is inserted only while `reserved < capacity`, and a unique partial index rejects a second active seat for the same user. Two requests cannot both observe spare capacity and both commit. The same file checks that a stale judge claim token does not apply a second verdict, and that a first solve is awarded once.

Our demo contest Last seat has capacity 1 and registration open. Two browsers register. One is seated and one is waitlisted. That is the same filter, with a room of one.

4 Security and the threat model

We keep five threats. Each row names the control we actually run, and the test that fails if the control is removed.

The hardening bot is inside the same model. It may read the problem and run the sandbox. `publish`, `edit_tests`, `delete_tests`, `change_limits`, `change_score`, and `disqualify` are denied. A proposal is held until we approve it. Section 13 is that path in full.

5 Performance, load, and the optimisation

The change we are comparing is the archive aggregation. Before it, the archive route ran one stats query per problem. After it, the archive is one aggregation. The before/after numbers are the

Table 5: Threats we designed for.

Threat	Control	Test
Submitted code escapes the host	`docker run`, capabilities dropped, read-only root, no network, a pid limit, user 65534. `/work` is read-only for the run.	isolation suite
A participant calls an admin route	The role check returns 403 and the denial is audited. The client also leaves the page.	auth, 403
A used refresh token is replayed	The refresh chain is revoked.	auth, replay
Registration oversells the room	The filtered seat update and the unique partial index.	400 against 200
Copied solutions are treated as independent	Winnowing similarity at 0.7 when a contest ends, and from Control. A match is a review. We do not disqualify automatically.	similarity

3 October run in `tests/load/results/k6.md`.

Table 6: Load, after the aggregation, and again on this laptop.

When	Route	p50 ms	p95 ms	Failed
3 Oct	archive	30.4	49.1	0%
3 Oct	leaderboard script	7.2	23.5	0%
5 Nov	archive	65.6	807.2	0%
5 Nov	leaderboard script	152.3	248.3	0%

On 3 October the archive held p95 49.1 ms at 99.9 requests/s, and the leaderboard script held p95 23.5 ms at 236.6 requests/s. Every threshold in the scripts passed.

On 5 November we ran the same scripts on this laptop while Docker, the judge, and the agent were also running. Throughput stayed near the target (99.6/s on `api`, 138/s on `leaderboard`) and no request failed. The latency thresholds on `api` were missed here (archive p95 807 ms). The leaderboard script stayed inside its 1000 ms threshold (p95 248 ms). We do not use the November numbers as the before/after of the aggregation. That comparison is the October table.

6 Failure and recovery

We ran `nodetests/failure/run.mjsredismongo` and `nodetests/failure/run.mjsjudge` against the stack that was already up. `JUDGE_RESTART=touch`, so `tsx watch` brought the judge back. Compose project `codeclash-srujan`.

```
PASS  redis: /health recovers
PASS  redis: the in-flight submission is judged once, judged AC
PASS  mongo: /health recovers
PASS  mongo: seat counts match active seats
PASS  mongo: new submissions are judged after the restart
PASS  judge: the claimed submission gets one verdict after restart, judged AC
```

What we require from a restart is one verdict, not two, and a seat count that still matches the active seats. Both held.

7 Defects

Jira project CC has five issues of type Bug. All five are Done. Earlier work was filed as stories. These five are the defect history.

Table 7: The five defects, each with the test that now fails if it returns.

Bug	What was wrong	Regression
CC-53	Compiled languages failed in CI. The compile container drops every capability, so a mode-755 temp directory rejected the binary and the verdict was CE.	languages, batch
CC-54	Concurrent registration could hand out more seats than capacity.	400 against 200
CC-55	A stale judge claim could award the same solve twice.	claim token
CC-56	A used refresh token could be replayed into a new session.	auth replay
CC-57	A participant could call an admin route.	403 and audit

CC-54, CC-55, CC-56, and CC-57 were already fixed on `main` when we opened this pack. CC-53 is pull request #47, merged 5 November 2026, with `check` and `e2e` green. CC-53 is the sandbox defect that made `check` red from pull request #40 onward. CC-54 and CC-55 are the concurrency defects. CC-56 and CC-57 are the security defects. The escape suite is the failure case for a hostile program, next to these five.

8 Cost

We estimated this pack at $0 of paid infrastructure before we wrote it. We run Docker on this PC. We call Gemini on the free tier. We have no judge fleet. We did not plan a NAT gateway, a load balancer, or RDS, because those bill immediately.

On 5 November we asked our account, `942893239786`, for its free-tier usage. The reply listed only AWS Glue. EC2 is not in our 12-month allowance, so a `t3.micro` would have billed us. We did not start one. The hours on that line are zero.

On 6 November the public demo needed a machine through 20 November. Oracle's Always Free Ampere shape was out of host capacity in Hyderabad, and a 1 GB micro cannot give the compiler its 512 MB. We started one paid shape on the $300 trial instead. The card is not charged while the trial lasts. The machine draws that credit. We delete it on 20 November. If it is still running when the 30-day trial ends, the card is charged.

Table 8: What we planned to pay, and what we paid.

Item	Our plan	Paid
This PC: Docker, Mongo, Redis, Mailpit	local, no invoice	$0
Gemini, `gemini-3.6-flash`	free tier, keys only in gitignored env	$0
One `t3.micro`, 20 GB gp3, for the slot, then stop	only if EC2 were still in the allowance	$0
Oracle `E5.Flex`, 1 OCPU, 8 GB, 6–20 Nov	trial credit; delete on 20 Nov	about $15 of $300
NAT gateway, load balancer, RDS	excluded	$0

The card has not been charged. The Hyderabad machine draws the trial credit, about $15 if we leave it up until 20 November, and we delete it that day. Unused credit expires with the trial. The commands we would run on an account that still has the EC2 allowance, including `aws ec2 stop-instances`, are in `deploy/README.md`.

9 Timeline

Deliverable 2 dissolved the fourth sprint. We are not planning one. The bars are our merge dates on `origin/main`.

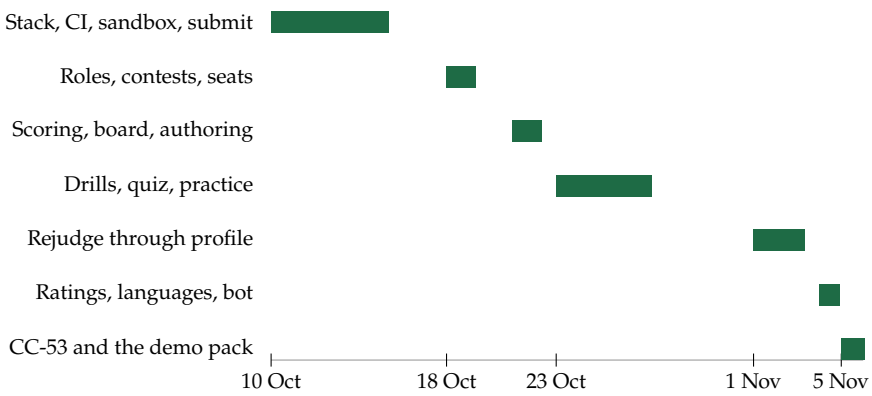


Figure 1: Our merges, from 10 October to 5 November. There is no fourth sprint after these bars.

Table 9: What we merged, from `git log --first-parent`.

When	What we merged
10–15 Oct	Local stack, modules, CI, sandbox, sign-in, submit, exactly-once judging, audit log (#4–#13).
18–19 Oct	Roles, contests, state machine, seats, waitlist, withdraw (#14–#19).
21–22 Oct	Contest submit, scoring, live board, authoring, tests, first solve (#20–#25).
23–27 Oct	Demo problems, failure drills, escape suite, backups, quiz, publish check, practice slot (#26–#32).
1–3 Nov	Rejudge, freeze, workers, audit search, profile (#33–#37).
4 Nov	Ratings (#38), metrics (#39), then #40–#46. `check` was red from #40.
5 Nov	#47, the compile permission. Similarity and mail are in that merge. #48, the demo pack.

10 Deployment

This section says what has to be running for CodeClash to work, what is running at `http://129.225.92.29`, and why that machine spends trial credit instead of sitting on Always Free.

10.1 What a running CodeClash is

A person opens the site in a browser. That page is not the whole system. Behind it, six other programs have to be alive at the same time.

1. The **page** is the React app. In development it is served by Vite at `http://127.0.0.1:5173`. On a server it would be ordinary files, served by Caddy.
2. The **server** answers the page. It is a Node program on port 4000. Sign-in, contests, submissions, and the live scoreboard all go through it. The live scoreboard is a Socket.IO connection that stays open. A host that only serves files cannot hold that connection.
3. **MongoDB** stores the users, the contests, the problems, and the verdicts. We run it as a replica set named `rs0` because a few of our writes need a transaction. It listens only on this machine.
4. **Redis** is the queue, not the database. When someone submits, the server does not run the code. It appends a message to a Redis stream. There are three streams: `judge:contest`, `judge:practice`, and `agent:harden`.
5. The **judge** is a second Node program. It takes one message off the stream and starts a throwaway Docker container. That container is the sandbox: no network, almost no permissions, a time limit, and then it is deleted. The judge needs the Docker socket of the same machine. It cannot live on a host that is not allowed to start containers.
6. The **agent** is a third Node program. It takes a message off `agent:harden` when a setter presses Check. It is the bot in §13. It can run without a Gemini key. With a key, it also asks the model for attacks.

7. **Mail** on this laptop is Mailpit. Signup sends a six-digit code, the page says the code is in the local inbox, and it links `http://127.0.0.1:8025`. SMTP is port 1025. On the public host the same page uses Oracle Email Delivery on port 587. A real relay returns no local link, so the page tells the person to open their own email.

So "deploy" here means one computer that runs all of these together. It does not mean uploading the React app to a static host. A static host cannot start a container for each submission, and it cannot keep the scoreboard socket open.

10.2 The public host

The address other people can open is `http://129.225.92.29`. It is one Ubuntu 24.04 machine in India South (Hyderabad), shape `VM.Standard.E5.Flex`, 1 OCPU and 8 GB. We started it on 6 November 2026. It is not Always Free. It spends the $300 trial credit. We delete it on 20 November 2026. If it is still running when the 30-day trial ends, the card is charged.

1. Docker runs MongoDB, Redis, and Mailpit. Each published port is bound to `127.0.0.1`. Caddy does not forward the inbox, and the API port stays closed to the internet.
2. Caddy on port 80 serves the built page and forwards `/api`, `/socket.io`, `/health`, and `/metrics` to the server on port 4000.
3. Three systemd units start the server, the judge, and the agent. The judge uses that machine's Docker. We pulled the six language images before the first submit.
4. The Ubuntu image allowed only SSH. We opened ports 80 and 443 and saved the rules, so a reboot keeps the site up.
5. Signup uses Oracle Email Delivery on port 587. The approved sender is the Gmail address we registered with Oracle. A register of that address returned `mailed: true`, with no code in the JSON and no local-inbox link. The server log has no send error, so the relay accepted the message. We have not opened that Gmail inbox. Gmail can still drop a `gmail.com` From address that Oracle signed.

We signed in on that page, opened "One number of FizzBuzz", and submitted Python. The page said Accepted. The first click did not. The site is plain HTTP, and `crypto.randomUUID` exists only on HTTPS and on localhost, so the button reported `crypto.randomUUID is not a function` and queued nothing. `newIdempotencyKey` falls back to `getRandomValues` on that page. After we rebuilt the client, the same submit was Accepted.

The two hardening files we open in the slot are already saved, so we do not wait for a new model call: `Deliverables/demo/hardening.json` and `Deliverables/demo/hardening-empty.json`.

10.3 Why this is not an Always Free machine

Oracle's Always Free Ampere shape, `VM.Standard.A1.Flex`, is the one that can hold this stack. The limits call on 6 November said the quota was unused. Every launch in Hyderabad still returned "Out of host capacity": 2 OCPU and 12 GB, 1 OCPU and 6 GB, each fault domain, and no fault domain. Quota is not a free host. `VM.Standard.E4.Flex` at 1 OCPU and 8 GB was out of capacity as well. The E5 shape at that size started.

A 1 GB machine does not fit the judge we ship. The compile container asks for 512 MB and turns swap off, and the Java floor is 512 MB, beside the operating system, Docker, and Mongo. The Always Free AMD micro is that 1 GB size. We did not shrink the judge to make it fit. Google's always-free `e2-micro` and Azure's free B-series are 1 GB too, and a static host cannot start a container per submission. Our AWS account still lists only Glue, so an EC2 instance would have billed the card directly. The trial credit is what carries the demo until 20 November. The steps and the delete

date are in `deploy/README.md`.

10.4 This PC, if the network fails

1. `docker compose up -d` starts MongoDB, Redis, and Mailpit. The compose project name is `codeclash-srujan`. A second compose project on this PC is named `codeclash`. We do not restart that one.
2. `npm run dev` starts the server, the judge, the agent, and the page.
3. If the database is empty, `npm run seed` fills it. The seeded password is `codeclash`.
4. We open `http://127.0.0.1:5173`. The sign-up page links the local inbox.
5. The Gemini key, if we use one, sits in `agent/.env`. That file is gitignored. We do not commit it and we do not print it.

10.5 The Amazon machine we wrote down and did not start

We also wrote the steps for putting that same set of programs on one small Amazon computer, so the demo could have an address other people could open. The computer would have been a `t3.micro`: two small virtual CPUs, 1 GB of memory, a public IP, and a 20 GB disk. Mongo, Redis, and Mailpit would still have been on that one machine, not on separate paid databases. Caddy, a small web server, would have served the built page and forwarded `/api` and the scoreboard socket to port 4000. Three systemd units would have started the server, the judge, and the agent when the machine booted. The judge would have used that machine's Docker.

We did not start it. On 5 November we asked our account, `942893239786`, which free-tier services it still had. The answer was only AWS Glue. EC2, which is the service a `t3.micro` belongs to, was not in the 12-month allowance. Starting the machine would have billed us. A NAT gateway, a load balancer, and RDS would have billed immediately even on a new account, so they were never part of the plan.

The commands, including `aws ec2 stop-instances` for after the slot, are in `deploy/README.md`. The files beside it are the Caddy config and the three boot scripts. They are the recipe. They are not a machine we are paying for.

11 Traceability

Each row is a requirement we can open, the issue that names it, the pull request that merged it, and the test that locks it.

Pull requests #40–#46 were merged with no review, and `check` was red from #40 because of CC-53. We did not rewrite those merges. #47 is what makes `check` able to go green. The row we say out loud is CC-53, pull request #47, `judge/src/languages.test.ts`. CC-35 and CC-40 are still To Do on the board even though the pull requests are merged. We record that in §12 rather than hiding it in this table.

12 Jira dashboard and project report

We queried `project = CC` again on 5 November 2026, after closing the issues whose work is already on `main`. The site is `https://cs455-codeclash.atlassian.net`. The board page asks for our Atlassian login, so the figure is the query, not a photograph of the site.

Seven epics had every child already Done, so we moved them to Done: CC-5 through CC-9, CC-11, and CC-12. CC-35, the test auditor, is the bot in §13, so we moved it to Done. CC-39 is role changes plus audit search, both on `main` (the role route, and pull request #36), so we moved it to Done.

Table 10: From a requirement to a test.

Requirement	Jira	PR	Test
Rejudge	CC-32	#33	flows
Freeze and reveal	CC-33	#34	flows
Workers	CC-38	#35	flows
Audit search	CC-39	#36	flows
Profile edits	CC-43	#37	auth
Ratings	CC-45	#38	flows
`/metrics`	CC-48	#39	metrics route
C, C++, Java, Go	none	#40	languages
Batch judge	none	#41	batch
IOI subtasks	none	#42	scoring
CodeContests import	none	#43	codecontests
Hardening engine	CC-35	#44	pipeline, ai
Hardening UI	none	#45	flows
k6 and smoke	CC-40	#46	tests/load
Compile permission	CC-53	#47	languages, batch
Similarity	CC-37	#47	similarity
Seats, 400 vs 200	CC-54	on main	domain
Stale claim	CC-55	on main	domain
Refresh replay	CC-56	on main	auth
Participant to admin	CC-57	on main	auth
Demo mail, seed, host notes	CC-58	#48	auth, seed



Figure 2: 58 issues. Bar length is the count. In progress is one issue, so that bar is the short one.

That is 49 Done, 1 in progress, and 8 to do. 58 in total. The path we can walk in the viva is still a requirement, the issue, the pull request, and the test, using §11.

13 The test-hardening bot

The bot is one worker. A setter opens Authoring, step Check, and starts a run. The server writes a document and appends the id to the Redis stream `agent:harden`. The worker claims that document with a token, the same way the judge claims a submission. It loads the statement, the input spec, the reference, and the tests. It then asks `providerFromEnv` whether a model is configured.

The two pipelines are the same until that question, and the same again once the sandbox starts. They differ only in whether a planner is attached.

13.1 Shared stages

Both pipelines run these stages. The model steps are the only ones that disappear when the key is absent.

1. Baseline. The reference runs on the tests we already stored. If it fails one, the note says so and we do not trust the score.
2. Mutants. Deterministic edits of the reference: a relational operator, an off-by-one, a dropped element.
3. Replay. Each mutant runs on the stored tests. One that fails a sample is trivial. One that fails a

Table 11: What is still open, and why it is still open.

Key	Status	Why it is open
CC-28	In progress	A per-user submission quota. Assigned to Peeyush Agarwal. We have not built it.
CC-36	To Do	Clarifications, and an agent that would triage them. We have not built them.
CC-40	To Do	A k6 burst of 400 registrations into 200 seats, and 600 submissions into 24 judge slots. We have the 400-against-200 test as a unit test, and k6 for the archive and the leaderboard. We did not run this burst.
CC-50	To Do	Global quotas for submissions, contests, and the agent budget. We have not built them.
CC-52	To Do	A Deliverable 2 branch of empty placeholder pages. We did not merge that branch. The real pages landed later, under stories that are already Done.
CC-10	To Do	The submission epic. It stays open because CC-28 is still in progress.
CC-13	To Do	The agents epic. The test auditor is Done. It stays open because CC-36 is not built.
CC-14	To Do	The admin epic. CC-39 is Done. It stays open because CC-50 is not built.
CC-15	To Do	The load-test epic. It stays open because CC-40 was not run.

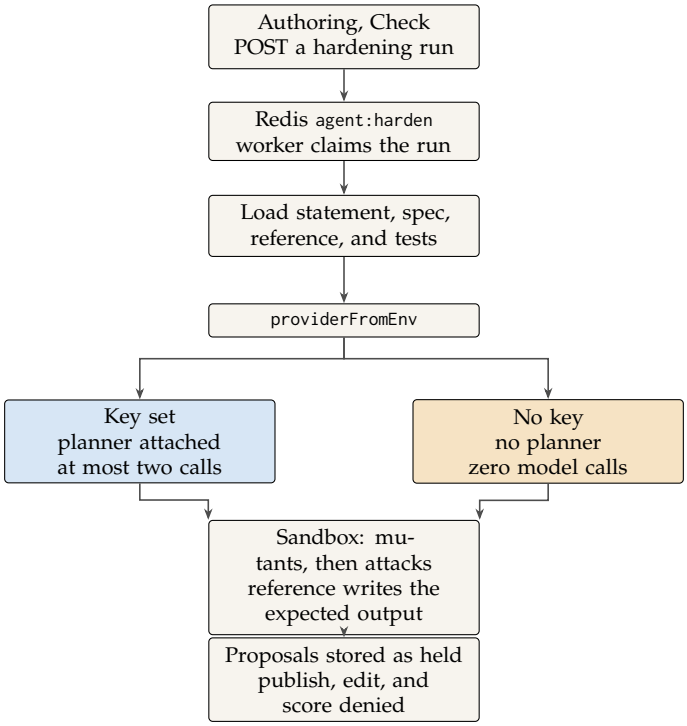


Figure 3: One queue and one sandbox. The key only decides whether a planner is attached.

hidden test is killed. One that still matches is survived.

4. Attacks from the spec: boundary, about 25 random inputs, and one max-scale input. With no spec we derive a few inputs from the shape of the samples, and we skip boundary and max-scale.
5. Model plan, only when a planner is attached. At most one call. The reply may contain a wrong program, an edge-case input, or a slow program. A `tool` field the gate denies is dropped and audited.
6. The reference runs every new input. A crash, an empty answer, or an answer over 64 KB is dropped.
7. Survivors face every accepted attack.
8. Model feedback, only when a planner is attached and something still survived. One more call. That is the second and last call of the run.
9. A deterministic mutant that survived the suite and at least 25 random inputs is marked equivalent.

We do not propose a test for it.

10. Proposals. The shortest input that kills a survivor. A wrong program the current tests already catch is held, not added again. If the largest stored test is under half the maximum input size, we also propose the max-scale input.
11. Score. Killed deterministic mutants divided by those that are killed or still survived. Equivalent edits and compile errors are out of the denominator. The score is null when nothing is eligible.

Table 12: Where the two pipelines differ.

Stage	With a key	Without a key
Provider	`AGENT_PROVIDER=gemini`, model `gemini-3.6-flash`, thinking off, 60 s limit. Status AVAILABLE.	Unset, `none`, or `fake`. Status DISABLED. Note: "No AGENT_PROVIDER is set."
Planner	Attached. At most two calls: plan, then feedback.	Omitted. Zero calls, zero tokens.
Model attacks	Wrong programs and edge inputs, if the reply parses.	None. The attack count for `model` is 0.
Spec attacks and mutants	Run.	Run. The note begins "AI-assisted analysis unavailable."
Publish, edit, score	Denied.	Denied.

A program the model offers that matches the reference on every input is discarded with the note that it may be correct. We do not store it as a wrong solution.

13.2 With a key

We saved this run so the demonstration does not wait on a fresh call. Problem: Larger of two, the published one. File: `Deliverables/demo/hardening.json`. Run `6aec8cfa3eb8ea40a4bcac1d`. Status completed, `aiStatus` AVAILABLE, 2 calls, 1089 tokens. Attacks: boundary 4, random 25, max 1, shape 0, model 1. Notes empty.

The only deterministic edit, >= changed to >, matched the reference, so it is equivalent and the score is null. The model still earned its place. Two wrong programs were killed by tests we already store, and both are held:

- Absolute values. The contestant assumes the inputs are positive.
- Lexicographic comparison. The contestant compares the inputs as strings.

On a suite that is already closed, a key does not invent a new test for an equivalent edit. It holds the wrong programs the suite already catches. The next subsection is the case where the suite is not closed, which is where a new test appears.

13.3 Without a key

Same problem, provider unset. The file is `Deliverables/demo/hardening-empty.json`.
Run `6aec8e113eb8ea40a4bcac22`. Status completed, `aiStatus` DISABLED, 0 calls, 0 tokens. Attacks: boundary 4, random 29, max 1, shape 0, model 0. Proposals: none.

The notes are "AI-assisted analysis unavailable. Running deterministic hardening suite." and "No AGENT_PROVIDER is set." The >= to > edit is still equivalent. The deterministic half of the pipeline ran. The model half did not. The random count is not the same number as the keyed run because the seed is the run id. We do not read that difference as an effect of the key.

This file is the one we repeat if the model is slow. It finishes in seconds.

13.4 Building three suites

A finished suite cannot show the bot adding a test, because every wrong program is already caught. We copied three problems into new drafts and deleted every test except the sample. We ran the pipeline, saved the short input it asked for, and ran it again. The published problems were not edited. The drafts are local, titled `Build: Larger of two`, `Build: Balanced brackets`, and `Build: Maximum subarray`.

13.4.1 Larger of two

The problem is: read two integers and print the larger one. If they are equal, print that value. The reference is one comparison, `a >= b`.

We started with the sample only. The input is `3 1` and the output is `3`. The first number is already the answer, so a program can ignore the second number and still pass.

Two programs the model wrote survived that sample.

```
print(a if abs(a) >= abs(b) else b)
```

On `3 1` the absolute values are 3 and 1, so it prints 3, which matches. It is wrong as soon as both numbers are negative, because it compares sizes and then prints the original number it thinks is larger.

```
ans = 0
if a > ans: ans = a
if b > ans: ans = b
print(ans)
```

On `3 1` both numbers are above 0, so it prints 3. If both numbers are negative it never moves off 0, and it prints 0.

The run asked for `-100 -50`, with expected output `-50`.

Table 13: Larger of two. What each program prints.

Input	Reference	Absolute value	Start at 0
`3 1`	3	3	3
`-100 -50`	−50	100	0

We saved `-100 -50`. We also added the two cases a setter writes next to a sample like this: `1 3` so the second number has to win, and `7 7` so equal values are accepted. On the next run the model came back with `print(max(abs(a), abs(b)))`. The line we had just saved already kills it: the absolute values are 100 and 50, and the program prints 100. The run did not ask for another correctness test.

Changing >= to > is still equivalent. When the two numbers differ, both comparisons pick the same one. When they are equal, printing either one is correct. The score stays empty because there is no remaining wrong edit to catch. That is the right result.

13.4.2 Balanced brackets

The problem is a string of parentheses. Print `yes` if they are properly nested, and `no` otherwise. The reference walks the string with a depth counter. `(` adds one, `)` subtracts one. The string is balanced only when the depth never goes negative and ends at zero.

```
print("yes" if ok and d == 0 else "no")
```

The sample is `(())`, and the answer is `yes`. Depth goes 1, 2, 1, 0. It never goes negative, and it ends at 0, so both halves of the condition are true.

The edit that survived flips `and` to `or`.

```
print("yes" if ok or d == 0 else "no")
```

On `(())` both halves are still true, so `or` also prints `yes`. The sample cannot see the bug. The score was 0, and the projected score after the proposed test was 1.

The shortest input the run asked for is a single `(`, expected `no`. Depth ends at 1, and it never went negative, so `ok` is true and `d == 0` is false. `and` prints `no`. `or` prints `yes`.

We saved that line. We also added two neighbours a setter would write once the sample is only the happy path:

- `)(` → `no`. One open and one close, so a program that only counts them says `yes`. The close comes first, so the depth goes negative.
- `((` → `no`. Two opens and one close. The string ends still open.

The next run killed the `and`/`or` edit. The score was 1. A different program still survived. It does not track depth at all. It says `yes` when four cheap checks pass: the length is even, the string starts with `(`, it ends with `)`, and the two counts are equal.

```
if (len(s) % 2 == 0 and s.startswith("(")
    and s.endswith(")") and s.count("(") == s.count(")")):
    print("yes")
```

`)(` kills a pure counter, because that program does not require the string to start with `(`. This one does, so `)(` is not enough. The run asked for `())(()`, expected `no`. Length 6, starts with `(`, ends with `)`, three opens and three closes, so the cheap checks all pass and the program prints `yes`. The depth goes 1, then 0, then −1. The reference prints `no`. We saved that string. The following run killed the program.

What the run still asked for, and what we did not add, is a long string. One model program is correct and slow: it deletes `()` until the string is empty, which is quadratic. It matches the reference on every short test. Timing it out needs a string of about 64,000 characters. That is a time-limit test, not a wrong answer. The correctness cases are done.

13.4.3 Maximum subarray

The problem is an array. Print the largest sum of a contiguous piece. The piece cannot be empty, so if every number is negative the answer is the largest of them, which is still negative.

The sample is

```
5
-2 1 -3 4 -1
```

and the answer is 4. The best piece is the single element `4`. The leading `-2` is not in that piece, and the trailing `-1` is not either. A program that drops the first element still sees `1 -3 4 -1` and still prints 4. A program that drops the last element still sees `-2 1 -3 4` and still prints 4. Six edits of that kind survived the sample. The score was 0. The projected score was 1.

The run offered a random triple, `373370295 -80059572 999066142`, and a one-element array whose only value was `-583680946`. Those inputs do kill the edits. We did not keep the random numbers. The same two holes have small tests, and those are the ones we saved.

The next run killed all six edits, and the score was 1. The only proposal left was a 62,500-element

Table 14: Maximum subarray. The two tests that replace the random triple.

Input	Answer	Why the sample did not catch it
`1 / -5`	−5	The only legal piece is −5. A program that starts its answer at 0 prints 0.
`2 / 1 100`	101	Both numbers are required. Dropping either one prints 1 or 100.

array, because our largest stored test has two numbers and the limit is about 200,000. The reference solved that large input in 200 ms of a 2000 ms limit. We did not add it. It is about the time limit, the same way the long bracket string is.

In Authoring, as Meghna, filter for `Build:`. The short demonstration is the first run of each draft: the sample, the program that still passed it, and the one line the run proposed.

14 AI engineering log

These are the interactions in which a recommendation had to be decided for this deliverable. Other sessions did ordinary drafting, and those are not included. A prompt is the request as sent. A reply is summarised. Each entry then records the evaluation, the error, the change, the artifact, and the reason. This is the same shape as the prompt log in Deliverable 2.

Table 15: Log entries in order.

No.	Stage	Date	Decision
L1	On main	5 Nov	The compile directory is writable. The run stays unprivileged.
L2	On main	5 Nov	The hardening agent may read and run. It may not publish, edit, or score.
L3	On main	5 Nov	Signup mail is a 6-digit code. The long token is only the local fallback.
L4	Not launched	5 Nov	One VM is the host shape. This account would be billed, so none was started.
L5	Local drafts	5 Nov	Three suites start from the sample. The closed-suite report is discarded.

14.1 L1. Sandbox permission

On main · 5 November 2026 · Om Chandrakant Chaudhari, reviewed by the team · Modified

1. Prompt

Compiled languages fail in CI. The sandbox drops every capability. Find the smallest change that lets the compiler write the binary, and keep the run unprivileged.

2. Reply (summarised)

The model first suggests putting `CAP_DAC_OVERRIDE` back on the compile container. The follow-up, after the failing test is shown, is to leave capabilities dropped and make the private temp directory mode `0777` before compile, then mode `0755` on the contents after a successful compile, so user 65534 can execute.

3. Decision

- **Evaluation.** The directory mode fixes the compile error without handing the container a capability.
- **Error.** Restoring `CAP_DAC_OVERRIDE` would undo the drop-all rule for every compile, including a hostile one.
- **Change.** `openWorkDir` chmod's the temp directory, then the compiled files. The run stays `-user 65534` and `/work` read-only.
- **Artifact.** `judge/src/runner.ts`. Jira CC-53. Pull request 47.
- **Reason.** C, C++, Java, and Go then accept a sum program. A fork bomb is still contained as RE or TLE.

14.2 L2. Agent boundary

On main · 5 November 2026 · Om Chandrakant Chaudhari, reviewed by the team · Kept

1. Prompt

Settle the test-hardening agent so a demo can show a result twice. The model is optional. Publish, edit, and score stay denied.

2. Reply (summarised)

The model proposes a second agent that may publish a repaired test when the score is low. It also proposes `gemini-2.5-flash`. A probe of the key returns 404 for that model and 200 for `gemini-3.6-flash`. With thinking left on, the call exceeds the old 20-second timeout. With thinking off and a 60-second limit, the call returns text.

3. Decision

- **Evaluation.** One worker, one tool list, and a saved run are enough for the slot.
- **Error.** A publish tool, and a model name the new keys cannot call.
- **Change.** Tools may read the problem and run the sandbox. A `publish` call is denied and audited. The model is `gemini-3.6-flash` with thinking off. Keys stay in gitignored env. `scripts/demo-harden.mjs` writes `Deliverables/demo/hardening.json`. A second run, with the provider unset, writes `Deliverables/demo/hardening-empty.json` and finishes in seconds. The slot opens the saved file.
- **Artifact.** `agent/src/engine/ai.test.ts`, `scripts/demo-harden.mjs`, the two JSON files.
- **Reason.** The deny is a test, not a slide. The empty-provider file is what we repeat if the model is slow.

14.3 L3. Six-digit mail

On main · 5 November 2026 · Om Chandrakant Chaudhari, reviewed by the team · Modified

1. Prompt

Signup in the demo should be the email code we already fought with, not a long token pasted from the page.

2. Reply (summarised)

The model keeps the long token and adds a mailer that also prints it. The follow-up is the code shape already used in the other tree: six digits, fifteen minutes, Mailpit on SMTP 1025.

3. Decision

- **Evaluation.** A code a person can type is the thing the slot can show. Mailpit is local and costs nothing.
- **Error.** Returning the long token whenever mail is configured trains the demo on a path production hides.
- **Change.** `newVerifyToken` is a six-digit code, valid for fifteen minutes. The token is returned only when SMTP is unset and `NODE_ENV` is not production. Auth tests still post whatever the API returns.
- **Artifact.** `server/src/domain/product.ts`, `server/src/mail.ts`, `client/src/pages/Auth.tsx`, Mailpit in `docker-compose.yml`.
- **Reason.** A local register returned `mailed: true`, Mailpit held the code, and `POST /api/auth/verify` returned 200.

14.4 L4. One VM

Not launched · 5 November 2026 · Om Chandrakant Chaudhari, reviewed by the team · Rejected for this account

1. Prompt

The live demo runs on one AWS VM. This PC is the backup. Do not add a NAT gateway, a load balancer, or RDS. If a `t3.micro` would bill, stop.

2. Reply (summarised)

The model lists a public `t3.micro`, 20 GB gp3, ports 22 and 80, Caddy, and three systemd units. It treats a missing free-tier row as "still free" and offers to launch.

3. Decision

- **Evaluation.** The shape is right: one machine, Mongo and Redis on localhost, the judge on that machine's Docker socket.
- **Error.** `get-free-tier-usage` on 5 November lists only AWS Glue. EC2 is not in the 12-month allowance. Launching would bill.
- **Change.** No EC2 instance was started. `deploy/README.md` keeps the commands, including `aws ec2 stop-instances`, for an account that still has the allowance. The public demo on 6 November is the Oracle trial machine in §10, not this shape.
- **Artifact.** `deploy/Caddyfile`, the three unit files, `Deliverables/demo-script.md`.
- **Reason.** This account stays at $0 of EC2. A billed instance left running would exhaust the course cap. The machine we did start is the Oracle trial host, and we delete that one on 20 November.

14.5 L5. Building a suite from the sample

Local drafts · 5 November 2026 · Om Chandrakant Chaudhari, reviewed by the team · Modified

1. Prompt

Document the hardening pipeline on harder, average problems already in the database. Include the mutant source, what was discarded, and which test was hardened. Then: scratch that. Take three problems, start their tests, and rerun the pipeline as the tests grow, so the demonstration shows the suite being built.

2. Reply (summarised)

The first reply runs five medium problems whose suites are already complete: Climbing stairs, Balanced brackets, Maximum subarray, Pair with a given sum, and Range sum. Every wrong program is already killed. The score does not move, and no test is added. The write-up says the suites were finished.

3. Decision

- **Evaluation.** A closed suite can show that a wrong program is held. It cannot show a test being added.
- **Error.** Reporting "nothing to add" as the result of the bot.
- **Change.** Three new drafts, copied from Larger of two, Balanced brackets, and Maximum subarray, each reduced to the sample. The next tests are the short lines the run asks for, plus the obvious neighbour. The random triple and the 64,000-character strings are not saved. The published problems are not edited.
- **Artifact.** `Deliverables/demo/hardening-build.md` and `hardening-build.json`. The walk-through is §13.

- **Reason.** On Larger of two the run names `-100 -50`. On brackets it names `(`, then `())(()`. On the subarray, `-5` and `1 100` replace the random triple and the score goes from 0 to 1.

`Deliverables/demo-script.md` is the ten minutes: one submit to AC, the saved keyed run and the empty-provider file, Last seat in two browsers, a Redis restart with the PASS lines as backup, the admin page refused to a participant, and the CC-53 row.